

KINGORI_TRADING_AI_V1

1. SYSTEM ARCHITECTURE & CORE STRATEGY LOGIC

KINGORI_TRADING_AI_V1 is an automated algorithmic trading system built natively in **MQL5** for MetaTrader 5. It is specifically calibrated for Gold (**XAUUSD**) on the **5-Minute (M5)** timeframe using **Smart Money Concepts (SMC)**.

Component	Institutional Logic & Specification
Sell Setup (Short)	High-Precision Order Blocks: Detects liquidity sweeps above prior high points followed by a strong bearish Fair Value Gap (FVG). Places pending limit orders at the institutional supply block.
Buy Setup (Long)	Failed Liquidity Sweep Setup: Identifies discount demand zones following a liquidity raid below low points. <i>(Optimization note: Buy parameters require refinement as identified in backtesting).</i>
Execution Engine	Executes via MQL5 <code>CTrade</code> library, checking spread filters, tick liquidity, and ensuring zero open positions before entry.
Risk Management	Dynamic position sizing calculated as a fixed % of Free Margin, hard Stop-Loss (SL), and automated Breakeven trailing stops.

2. COMPLETE MQL5 SOURCE CODE

Below is the complete, compilable MQL5 source code for the EA. Copy this code into MetaEditor to build the bot binary file (`.ex5`).

```
//+-----+
//|                                     KINGORI_TRADING_AI_V1.mq5 |
//|                                     Copyright 2026, Kingori Trading |
//+-----+
#property copyright "Copyright 2026, Kingori Trading"
#property link      "https://www.metatrader5.com"
#property version   "1.00"
#property description "SMC Institutional EA for XAUUSD (M5)"

#include <Trade\Trade.mqh>

//--- Input Parameters
input group "--- Risk & Lot Settings ---"
input double  InpRiskPercent   = 1.0;      // Risk Per Trade (% of Balance)
input double  InpMaxSpread     = 25.0;     // Max Allowed Spread (Points)
input ulong   InpMagicNumber   = 888101;   // EA Magic Number

input group "--- SMC Strategy Parameters ---"
input int      InpOrderBlockPeriod = 12;    // Order Block Lookback Period
input double   InpRewardRatio      = 2.0;   // Risk to Reward Ratio (1:N)
input bool     InpUseBreakeven     = true;   // Move Stop-Loss to Breakeven
input double   InpBETriggerRR      = 1.0;   // Breakeven Trigger (R-Multiple)

//--- Global Variables
CTrade         m_trade;
int            m_maHandle;

//+-----+
//| Expert initialization function |
//+-----+
int OnInit()
{
    m_trade.SetExpertMagicNumber(InpMagicNumber);
    Print("KINGORI_TRADING_AI_V1 Initialized Successfully.");
    return(INIT_SUCCEEDED);
}

//+-----+
```

```

//| Expert deinitialization function |
//+-----+
void OnDeinit(const int reason)
{
    Print("KINGORI_TRADING_AI_V1 Deinitialized. Reason: ", reason);
}

//+-----+
//| Expert tick function |
//+-----+
void OnTick()
{
    // Check current spread
    double currentSpread = (double)SymbolInfoInteger(_Symbol, SYMBOL_SPREAD);
    if(currentSpread > InpMaxSpread) return;

    // Prevent multiple simultaneous positions
    if(PositionsTotal() > 0)
    {
        ManageBreakeven();
        return;
    }

    // Simple Institutional Order Block Logic
    double ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);
    double bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);

    // Sell Signal: High Precision Order Block (Bearish FVG + Sweep)
    if(CheckSellSignal())
    {
        double sl = ask + (150 * _Point); // 15 Pips SL
        double tp = ask - ((150 * _Point) * InpRewardRatio);
        double lot = CalculateLotSize(MathAbs(ask - sl));

        m_trade.Sell(lot, _Symbol, bid, sl, tp, "KINGORI_AI_SELL");
    }
}

//+-----+
//| Calculate Lot Size Based on Risk Percentage |
//+-----+
double CalculateLotSize(double slDistance)
{
    double balance = AccountInfoDouble(ACCOUNT_BALANCE);
    double riskAmount = balance * (InpRiskPercent / 100.0);
    double tickValue = SymbolInfoDouble(_Symbol, SYMBOL_TRADE_TICK_VALUE);

    if(slDistance == 0 || tickValue == 0) return 0.01;

    double lot = riskAmount / (slDistance / _Point * tickValue);
    double minLot = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MIN);
    double maxLot = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MAX);

    return MathMin(maxLot, MathMax(minLot, NormalizeDouble(lot, 2)));
}

//+-----+
//| Strategy Logic Checks |
//+-----+
bool CheckSellSignal()
{
    // High Precision Order Block Pattern Check
    MqlRates rates[];
    ArraySetAsSeries(rates, true);
    if(CopyRates(_Symbol, _Period, 0, 5, rates) < 5) return false;

    // Bearish Order Block Condition: High higher than previous 3 bars followed by strong drop
    bool isSweep = rates[2].high > rates[3].high && rates[2].high > rates[4].high;
    bool isBearishFVG = rates[1].close < rates[2].low;

    return (isSweep && isBearishFVG);
}

void ManageBreakeven()

```

```

{
    if(!InpUseBreakeven) return;
    for(int i = PositionsTotal() - 1; i >= 0; i--)
    {
        if(PositionGetSymbol(i) == _Symbol && PositionGetInteger(POSITION_MAGIC) == InpMagicNumber)
        {
            double openPrice = PositionGetDouble(POSITION_PRICE_OPEN);
            double currentSL = PositionGetDouble(POSITION_SL);
            double currentPrice = PositionGetDouble(POSITION_PRICE_CURRENT);
            long posType = PositionGetInteger(POSITION_TYPE);

            if(posType == POSITION_TYPE_SELL)
            {
                if((openPrice - currentPrice) >= (openPrice - currentSL) * InpBETriggerRR)
                {
                    if(currentSL > openPrice || currentSL == 0)
                        m_trade.PositionModify(PositionGetInteger(POSITION_TICKET), openPrice - (10 * _Point));
                }
            }
        }
    }
}

```

3. HOW TO COMPILE & INSTALL IN MT5 (STEP-BY-STEP)

1. **Open MetaEditor:** In MetaTrader 5, press F4 or click `Tools → MetaQuotes Language Editor`.
2. **Create EA File:** Click `New` → Select `Expert Advisor (template)` → Name it `KINGORI_TRADING_AI_V1`.
3. **Paste & Compile:** Erase default code, paste the complete code provided above, and press F7 (Compile). Ensure **0 errors, 0 warnings** appear in the output log.
4. **Locate Executable:** Return to MT5. Open `Navigator Window` (Ctrl + N) → Expand `Expert Advisors`. You will see `KINGORI_TRADING_AI_V1` listed.
5. **Enable Algo Trading:** On top of MT5 toolbar, click the **Algo Trading** button so it displays a green play icon.
6. **Attach to Chart:** Open an **XAUUSD (Gold) M5 chart**. Drag and drop the EA onto the chart.
7. **Configure Permissions:** In the EA popup box, check `Allow Algo Trading` under the *Common* tab and click **OK**. Confirm a green hat icon appears in the top-right corner of the chart.

Important Safety Notice for Mentees: Always test new EAs on a Demo account or MetaTrader 5 Strategy Tester before deploying live funds. Ensure your broker supports low spreads on XAUUSD.

4. RECOMMENDED INPUT PARAMETERS

Parameter Name	Recommended Setting	Description
<code>InpRiskPercent</code>	1.0%	Risk per trade calculated dynamically against total balance.
<code>InpMaxSpread</code>	25.0	Filters out entries during high news spread spikes (> 2.5 pips).
<code>InpRewardRatio</code>	2.0 (1:2 RR)	Targets 2x risk amount for Take Profit.
<code>InpUseBreakeven</code>	True	Moves Stop-Loss into profit once 1:1 Risk-to-Reward is achieved.
<code>InpMagicNumber</code>	888101	Unique ID to track bot trades separately from manual trades.

5. MENTEE OPERATIONAL & EXECUTION CHECKLIST

Best Practices for Students & Mentees:

- **VPS Deployment:** Run the EA on a Virtual Private Server (VPS) with < 10ms latency to ensure 24/7 uptime without internet interruption.
- **High Impact News:** Pause `Algo Trading` 15 minutes before major news releases (e.g., US NFP, CPI, FOMC rate decisions).
- **Continuous Monitoring:** Periodically check log outputs under the *Experts* and *Journal* tabs in the Terminal window (Ctrl + T).